

```

# from jupyter_dash import JupyterDash
from business import GraphBuilder, StatsBuilder
from dash import Input, Output, State, dcc, html, Dash

# Task 7.4.1
from dash import Dash
app = Dash(__name__)

# Task 7.4.8
gb = GraphBuilder()

# Task 7.4.13
sb = StatsBuilder()

# Tasks 7.4.1, 7.4.2, 7.4.3, 7.4.11, 7.4.14, 7.4.16
app.layout = html.Div(
    [
        html.H1("Application Demographics"),

        dcc.Dropdown(
            options=["Nationality", "Age", "Education"],
            value="Nationality",
            id="demo-plots-dropdown"
        ),

        html.Div(id="demo-plot-display"),

        html.H1("Experiment"),
        html.H2("Choose your effect size"),
        dcc.Slider(min=0.1, max=0.8, step=0.1, value=0.2, id="effect-size-slider"),
        html.Div(id="effect-size-display"),
        html.H2("Choose experiment duration"),
        dcc.Slider(min=1, max=20, step=1, value=1, id="experiment-days-slider"),
        html.Div(id="experiment-days-display"),
        html.H1("Results")
        html.Button("Begin Experiment", id="start-experiment-button", n_clicks=0),
        html.Div(id="results-display")
    ]
)

# Tasks 7.4.4, 7.4.8, 7.4.9, 7.4.10
@app.callback(
    Output("demo-plots-display", "children"),
    Input("demo-plots-dropdown", "value")
)

def display_demo_graph(graph_name):
    """Serves applicant demographic visualization.

    Parameters
    -----
    graph_name : str
        User input given via 'demo-plots-dropdown'. Name of Graph to be returned.
        Options are 'Nationality', 'Age', 'Education'.

    Returns
    -----
    dcc.Graph
        Plot that will be displayed in 'demo-plots-display' Div.
    """
    if graph_name == "Nationality":
        fig = gb.build_not_choropleth()
    elif graph_name == "Age":

```

```

        fig = gb.build_age_hist()
    else:
        fig = gb.build_ed_bar()
    return dcc.Graph(figure=fig)

```

Task 7.4.13

```

@app.callback(
    Output("effect-size-display", "children"),
    Input("effect-size-slider", "value")
)
def display_group_size(effect_size):
    """Serves information about required group size.

    Parameters
    -----
    effect_size : float
        Size of effect that user wants to detect. Provided via 'effect-size-slider'.

    Returns
    -----
    html.Div
        Text with information about required group size. will be displayed in
        'effect-size-display'.
    """
    n_obs = sb.calculate_n_obs(effect_size)
    text = f"To detect an effect size of {effect_size}, you would need {n_obs} observations."
    return html.Div(text)

```

Task 7.4.16

```

@app.callback(
    Output("experiment-days-display", "children"),
    Input("effect-size-slider", "value"),
    Input("experiment-days-slider", "value")
)
def display_cdf_pct(effect_size, days):
    """Serves probability of getting desired number of observations.

    Parameters
    -----
    effect_size : float
        The effect size that user wants to detect. Provided via 'effect-size-slider'.
    days : int
        Duration of the experiment. Provided via 'experiment-days-slider'.

    Returns
    -----
    html.Div
        Text with information about probability. Goes to 'experiment-days-display'.
    """
    # Calculate number of observations
    n_obs = sb.calulate_n_obs(effect_size)

    # Calculate percentage
    pct = round(sb.calulate_cdf_pct(n_obs, days), 2)

    # Create text
    text = f"The probability of getting this number of observations in {days} days is {pct}%"
    # Return Div with text
    return html.Div(text)

```

Task 7.4.18

```

@app.callback(
    Output("results-display", "children"),

```

```

    Input("start-experiment-function", "n_clicks"),
    Input("experiment-days-slider", "value")
)
def display_results(n_clicks, days):
    """Serves results from experiment.

    Parameters
    -----
    n_clicks : int
        Number of times 'start-experiment-button' button has been pressed.
    days : int
        Duration of the experiment. Provided via 'experiment-days-display'.

    Returns
    -----
    html.Div
        Experiment results. Goes to 'results-display'.
    """
    if n_clicks == 0:
        return html.Div()
    else:
        # Run experiment
        sb.run_experiemnt(days)
        # Create side-by-side bar chart
        fig = gb.build_contingency_bar()
        # Run chi-square
        result = sb.run_chi_square()
        # Return results
        return html.Div(
            [
                html.H2("Observations"),
                dcc.Graph(fig),
                html.H2("Chi-Square Test for Independence"),
                html.H3(f"Degrees of Freedom: {result.df}"),
                html.H3(f"p-value: {result.pvalue}"),
                html.H3(f"Statistic: {result.statistic}")
            ]
        )

```